

CBBA 出價參數

uav_cbba • scoring.cpp / energy_model.cpp / cbba_agent.cpp • 2026-10-05

每個參數為什麼放在公式的那個位置、目前的數值代表什麼，以及改了會怎樣。實驗數據來自 `cbba_sim run`（丟包 0%），場景是 `fire_3uav.csv` 和 `battery_demo.csv`。

先講結論

1. **電量參數是測試用的值，不是實測值**。每公尺 0.5%、懸停每秒 0.2% 是從舊版 `cbba_battery_test.py` 搬過來的。用這組值，`fire_3uav` 的 3 台無人機只做得完 8 個任務中的 5 個。
2. `cost_ref` 和 `battery_weight` 決定「遠的火點」和「近的例行點」誰優先。測了 25 組組合，其中 12 組的第二個火點沒有人接，三台都去做價值只有一半的例行點。
3. `battery_weight` 只在 0~2 之間有作用。超過 2 之後，兩個場景的分配結果都不再改變。

公式回顧

每段	$cost_j = Ls_j \times Lt_j$
	$Ls_j = \text{前一點到任務點的距離 (m)}$
	$Lt_j = \text{到達時間 / 剩餘期限} \quad (> 1 \text{ 表示逾期})$
路徑	$C_{ST} = \sum cost_j$
電量 不可行)	$B = E_{path} / (\text{battery} - \text{safety_reserve}) \quad (E_{path} \text{ 含回程}; > 1)$
總成本	$C_{total} = C_{ST} \times (1 + \text{battery_weight} \times B)$
出價	$\Delta = C_{total}(\text{插入後}) - C_{total}(\text{插入前}) \quad (\text{取所有插入位置中最小的})$
	$score = value / (1 + \Delta / cost_ref)$
	$bid = \min(score, \text{bundle 內既有出價的最小值})$

參數逐一說明

時空成本

$L_s \times L_t$

成本形式

為何這樣用

來自 Velhal 2022。兩者相乘，代表任務要「又遠又急」才真的貴。近的任務就算很急， L_s 小，成本也小；遠但不急的任務， L_t 小，成本也小。

單位

L_t 沒有單位，所以成本的單位是公尺。這也是 `cost_ref` 用公尺的原因。

附帶效果

到達時間是沿路徑累積的，排在後面的任務 L_t 自然比較大。所以插入位置會傾向把急的任務排前面。

`cruise_speed`

5.0 m/s

數值意義

等於 PX4 多旋翼預設的水平巡航速度

`MPC_XY_CRUISE`。場景檔可以替每台載具個別設定。

影響

只用來估計到達時間，也就是 L_t 。電量是以距離計算的，和速度無關。

`speed_margin`

0.8 → 規劃用 4 m/s

為何這樣用

加減速、轉彎、風和位置控制收斂，都會讓實際平均速度低於設定值。把到達時間估得晚一點，期限判斷會偏保守：寧可以為會遲到，也不要答應了卻趕不上。

數值意義

預留 20% 的速度餘裕，是經驗值。實機可以用 `u_log` 量「實際平均速度 ÷ 設定速度」，直接拿來當這個值。

`min_deadline`

1 s

為何這樣用

只是防止除以零。已經逾期的任務，剩餘期限會被夾在 1 秒，這時 L_t 等於到達秒數。成本會很大但仍是有限值，任務還是可以被接，這就是「軟限制」。

電量

**battery /
safety_reserve**
100 / 20 (%)

數值意義 電量以百分比表示。做完整條路徑並返航後，至少要剩 20%，否則路徑不可行。

為何是 20 PX4 的低電量警告門檻 `BAT_LOW_THR` 預設是 15%。規劃時預留 20%，正常飛完不會觸發 PX4 的低電量 failsafe，還留了 5% 給估計誤差。

**E / (battery -
reserve)**
電池因子 B

為何這樣用 用「可用電量」來正規化，不同電量的機子才比得公平。同樣耗 20% 電：對可用 35% (55-20) 的機子是用掉 57%，對可用 80% 的機子只是 25%。低電量的機子出價自然就低。

為何算回程 做完最後一個任務還要飛回 home。不算回程的話，規劃出來的路徑可能讓機子停在遠處、沒電回不來。

來源 這是本專案自己加的延伸。舊版 Python 的註解明寫它不是 Gul 2023 的原始公式。

**energy_per_meter
hover_energy_per_sec**
0.5 %/m · 0.2 %/s

數值來源 舊版 `cbba_battery_test.py` 的測試值，沒有實測依據。需要校正

換算 $0.5\%/\text{m} \times 5\text{ m/s} = \text{每秒 } 2.5\%$ ，是懸停 (0.2 %/s) 的 12.5 倍。滿電只能飛 200 m，卻能懸停 8 分鐘以上。實際的多旋翼前飛功率和懸停差不多，所以這組數字等於把場地縮小了，讓 100 m 的火場也碰得到電量限制。

實機參考 假設續航 15 分鐘，懸停約 0.11 %/s，以 5 m/s 前飛約 0.02 %/m，比目前的值小 20 倍以上。要用 `uLog` 的 `battery_status` 實際量。

合成與出價

C_ST × (1 + w·B)
乘法合成

為何這樣用 B 沒有單位，用乘的不需要另外找「公尺換電量」的係數。電量直接表示成「時空成本要放大幾倍」。

缺點 C_ST 接近 0 時（載具就在任務附近，或期限很長），不管多耗電成本都還是 0。這時電量只剩「不可行」的硬限制有作用。

battery_weight

1.0

數值意義 $w = 1$ 時，一條用光可用電量 ($B = 1$) 的路徑，成本會變成兩倍。也就是「耗電」和「時空成本」各占一半。

實測 0~2 之間會改變分配，超過 2 之後結果都一樣（見下方實驗）。合理的調整範圍就是 0~2。

邊際成本 Δ

插入前後的差

為何這樣用 出價要反映「多接這個任務會讓我的整條路徑變差多少」，而不是單獨做它的成本。這樣同一台機子接多個順路的任務會比較划算。

配套 插入式的邊際分數可能因為順路而「越接越高」，這會破壞 CBBA 的收斂保證。所以出價會被壓在 bundle 既有出價的最小值以下 (bid warping, Johnson 2012)。

value / (1 + Δ/c)

成本轉分數

為何這樣用 $\Delta = 0$ 時分數等於 value，是上限。成本越高分數越低，但永遠不會變成負數。如果用 value - Δ ，成本一大就會出現負分，而且 value 和 Δ 的單位也對不起來。

cost_ref

50 m

數值意義 邊際成本等於 50 時，分數剛好是 value 的一半。成本 50 相當於「飛 50 m，而且剛好在期限到」($L_s = 50$ 、 $L_t = 1$)。50 m 大約是 100×80 m 火場的半寬。

重要性質 對同一個任務，cost_ref 不影響誰贏：大家的 value 相同， Δ 小的分數一定高。它影響的是**不同價值的任務之間怎麼取捨**。

推導 火點 (value 80) 要贏過例行點 (value 40)，條件是 $\Delta_{\text{火}} < c + 2 \cdot \Delta_{\text{例}}$ 。cost_ref 越大，越願意為了高價值的任務多跑遠路。

value / deadline

火點 80 / 60 s

例行 40 / 300 s

數值意義 任務本身的權重：火點的價值是例行點的 2 倍，期限是 1/5。地面處置任務（火警）用 100 / 120 s。

注意 「2 倍」不足以保證火點優先，實際還要看 cost_ref，見下方實驗一。

共識穩定

score_epsilon
1e-3

- 為何這樣用 分數是用 float32 傳的 (`Bid.score`)。100 左右的 float32 精度約 1e-5，epsilon 比它大兩個數量級，各台機子比較同一組分數時，不會因為捨入誤差而得出不同結論。
 - 平手規則 分數差在 epsilon 內視為平手，由編號小的得標。所有機子用同一條規則，結果才會一致。
-

rebid_threshold
0.1

- 為何這樣用 這是滯後 (hysteresis)。機子在飛，位置和電量一直在變，分數每個週期都會小幅抖動。變動不到 10% 就不更新，可以避免任務在兩台之間來回換手，也就是 BT 被反覆中斷。
 - 數值意義 經驗值。調大會比較穩但反應慢，調小則反應快但容易抖動。
-

max_bundle
5

- 為何這樣用 每次插入要試 $n+1$ 個位置，每個位置都要重算整條路徑，建一次 bundle 大約是 $O(\text{任務數} \times n^3)$ 。上限 5 可以控制計算量，也限制規劃的視野：越後面的任務，位置和電量估得越不準。
-

實驗一：cost_ref $\times$ battery_weight

fire_3uav：3 台無人機，電量分別是 100、55、75。8 個空中任務：2 個火點 (value 80、期限 60 s、懸停 10 s) 和 6 個例行點 (value 40、期限 300 s、懸停 5 s)。每一格是一次完整協商，全部都在 0.14 s 內收斂。

第二個火點有沒有人接

格內數字是分配出去的任務價值總和

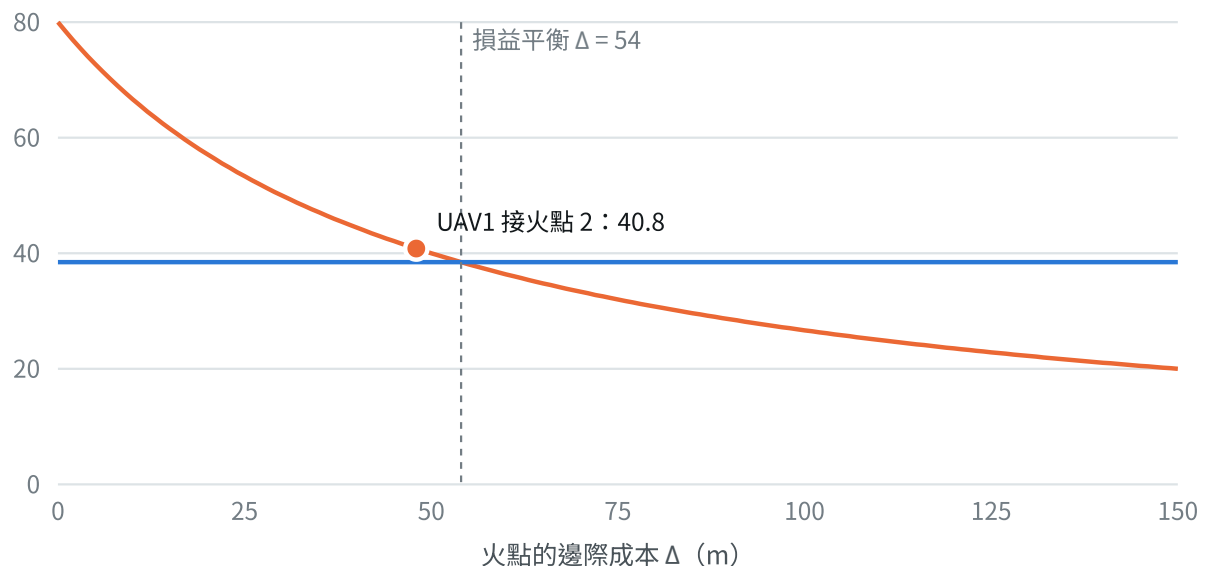
		cost_ref (m)				
		10	25	50	100	200
battery_weight	0	240	280	280	280	280
	0.5	240	240	280	280	280
	1	240	240	280	280	280
	2	240	240	240	280	280
	5	240	240	240	240	280

■ 兩個火點都有人接 (280) □ 火點 2 沒人接，改做例行點 (240)

cost_ref 越小、battery_weight 越大，越容易放棄火點 2。火點 2 在 (25, 10)，離有電接它的 UAV1 有 76 m。距離遠讓 Δ 變大，耗電又讓 Δ 再被電池因子放大，分數就掉到近處例行點以下。

火點能不能贏例行點：用 UAV1 的實際數字

UAV1 單獨接火點 2 的 $\Delta \approx 48$ ；單獨接例行點 3 的 $\Delta \approx 2$ ($w = 1$)



— 火點分數 $80 / (1 + \Delta/c)$ — 例行點 3 的分數 ($\Delta = 2$)

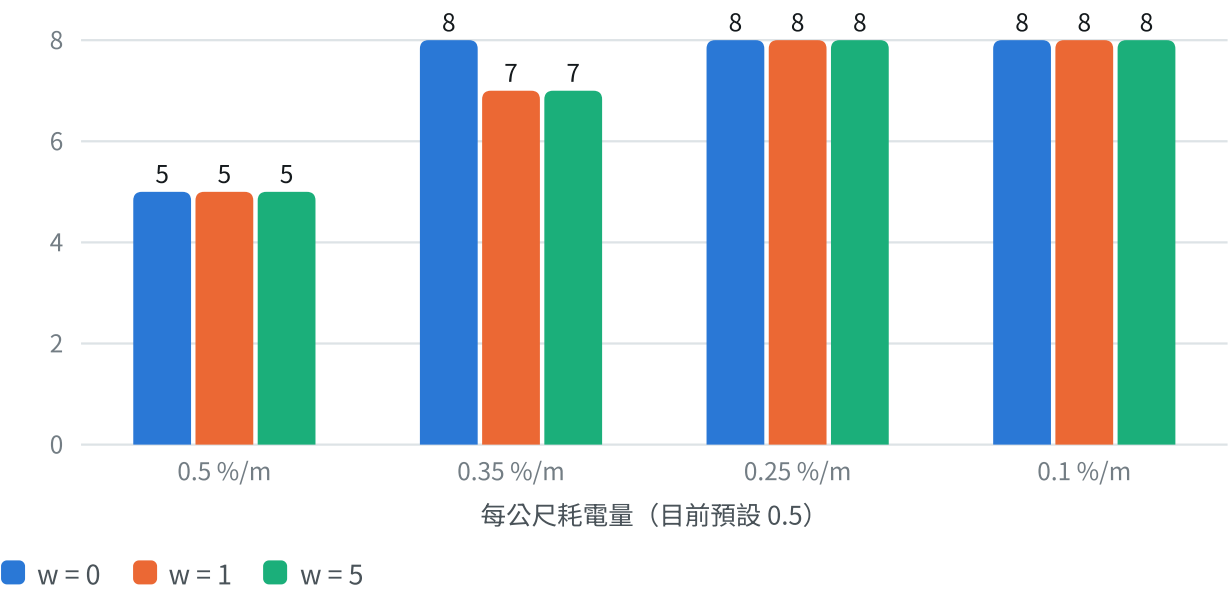
$c = 50$ ：火點 2 的分數 40.8，例行點 3 的分數 38.5，火點贏。

兩條線的交點就是損益平衡：火點的 Δ 超過 $c + 2 \cdot \Delta_{\text{例}}$ 就會輸。 $c = 10$ 或 25 時， $\Delta = 48$ 落在交點右邊，火點輸； $c = 50$ 或 100 時，火點贏。結果和實驗一 $w = 1$ 那一列吻合。

實驗二：每公尺耗電量

同一個 fire_3uav 場景，只改三台的 `energy_per_meter`，`cost_ref` 固定 50。

分配出去的任務數（共 8 個）



在 0.5 %/m 時，電量是最主要的瓶頸，8 個任務只做得完 5 個。降到 0.25 以下，全部都分得出去，這時電池權重才是在調「誰做」，而不是「做不做得到」。例如 0.1 %/m、w = 1 時，UAV1（100%）會從 UAV3（75%）那裡多接一個任務，兩台的可用電量用量分別是 21% 和 28%，比較平均。

實驗三：低電量機會不會讓出任務

battery_demo：UAV1 在 (-10, 0)，電量 34%（可用只有 14%）；UAV2 在 (12, 0)，電量 150%。3 個任務在兩台中間。

battery_weight	UAV1 任務數	UAV1 返航後電量	UAV2 任務數	說明
0 ~ 0.5	1	23.0	2	UAV1 接最近的中間任務（10 m）
1 ~ 20	1	20.2	2	中間任務被 UAV2 搶走，UAV1 改接較遠的邊緣任務，返航後只比安全存量多 0.2
50 ~ 100	0	34.0	3	UAV1 完全退出

電池權重只會改變「誰做比較划算」，不會讓低電量機放棄還做得到的任務。 $w = 1$ 時，UAV1 反而被推去做更耗電的任務。如果希望低電量機盡量留在後面，需要在接近安全存量時加更強的懲罰，例如可用電量的倒數形式，或是另設一個「保守門檻」。

建議

1. **先校正電量參數**。用實機或 SITL 的 `ulog`（`battery_status`、`vehicle_local_position`）量每公尺耗電和懸停每秒耗電。這是目前對分配結果影響最大的一組數字，而且現在的值和真實的差了一個數量級以上。
2. **決定火點要不要絕對優先**。如果要，有兩種做法。一是把火點的 `value` 拉開，例如 200 對 40。二是讓火點走獨立的規則：先分配火點，再分配例行點。單靠 `cost_ref` 撐住優先權，換個場地尺寸就會失效。
3. **`cost_ref` 跟著場地尺寸設定**。大約取場地半寬，或「一段典型航程」的長度。換到實機場地時要一起改。
4. **`battery_weight` 維持 1，只在 0~2 之間調**。更大的值沒有額外效果。
5. **考慮把電池項改成加法（選用）**。例如 $C_{ST} + w \times cost_ref \times B$ ，讓近距離的任務也會受耗電影響。這會改變分數尺度，改之前要重跑三個場景比對。

重現方式：`cbba_sim run <場景> --battery-weight W --cost-ref C`。實驗二是把場景檔的 `energy_per_meter` 欄位改掉後重跑。